WEEKLY PROGRESS REPORT -06

LIVE DATA LAYER: SUPABASE WIRED TO THE MARKETPLACE, DASHBOARDS & BIDDING

Project: MuktoKormo (Freelance Marketplace) Scope: Live Data & Persistence Wiring
Frontend: Next.js 16, React 19, Tailwind v4 Backend: Supabase (PostgreSQL, Auth, RLS)
Sprint: Week 6 (Live Data Wiring) Next Phase: Hiring Loop & Escrow Persistence
Team Members: Abdul Ahad Akid (2207018) | Ashraful Hosen (2207042) Status: Deployed and verified

1. Project Overview & Architectural Status

Executive Context: MuktoKormo is an Upwork/Fiverr-grade freelance marketplace engineered for the Bangladeshi digital ecosystem, connecting local employers with verified remote talent through a milestone-based escrow workflow. Earlier sprints delivered the complete front-end experience and a live Supabase backend with authentication. The interface, however, was still rendering static mock datasets.

Sprint Objectives: This sprint removed the mock foundation. Every marketplace surface that a real user touches now reads and writes live rows in the hosted PostgreSQL database, through a dedicated typed data-access layer, while the presentation layer and its component contracts were held completely intact.

Architectural Status: The hosted project (ref dhkhwaazyzikaoaontzn) remains the single source of truth. A new client-side data layer (src/lib/data) sits between React components and Supabase; components continue to receive and emit the same domain types they always did, so no UI was redesigned. All writes execute as the authenticated user and are therefore governed by Row Level Security - there is no service-role key in the application. Contracts and the wallet/ledger were deliberately deferred and continue to run on mock data, keeping the sprint tightly scoped to the marketplace, dashboards and bidding flows.

2. Schema Extensions in Detail

The base seven-table schema was extended - never replaced - so all existing constraints, foreign keys and the signup trigger continue to hold. New columns carry defaults where required, so rows created by the existing auth flow remain valid.

Table	Added Columns / Objects
projects	title_bn, description_bn, category, job_type, experience_level, timeline_days, budget_bdt, completion_pct, updated_at; expanded status check (draft, open, in_progress, completed, cancelled, archived, pending_review, expired); supporting indexes.
proposals	delivery_days, bidding_type, platform_fee, net_earned, payout_method, payout_account, portfolio_links, updated_at.
freelancers	location, avatar_url, rating, reviews_count, earned_bdt, success_rate, is_nid_verified, available, updated_at.
clients	location, avatar_url, rating, reviews_count, is_verified, updated_at.
proposal_milestones	New table: per-milestone breakdown of a milestone-based bid (title, amount_bdt,
saved_talent	

delivery_days,
sort_order) with party-scoped RLS.

New table: client bookmarks of freelancers, owner-only RLS, unique (client_id, freelancer_id).

3. Denormalization & Public Read Access

Two RLS realities made direct reads impossible for the public marketplace, and were resolved by denormalization rather than by weakening ownership rules:

· Display names. Names live in public.users, whose RLS only lets a user read their own row. A display_name
column was added to clients and freelancers, populated on insert and kept in sync by security-definer triggers, so the feed, bid room and directories can show other people's names.

· Proposal counts. Proposals are readable only by the two contracting parties, so the public feed cannot count them. A projects.proposals_count column is maintained by a trigger on proposal insert/delete/status
change, giving every visitor an accurate bid count.

authenticated users. Narrow anon SELECT policies were added for open (non-draft) projects, clients, and
freelancers; policies are OR-evaluated, so authenticated access is unchanged and nothing writable was
exposed.

All three additions are idempotent (IF NOT EXISTS / DROP IF EXISTS) and include backfills, so the migration is safe to
re-run and immediately correct for pre-existing rows.

4. Typed Data-Access Layer

A single module boundary (src/lib/data) now owns every database interaction. Components never build queries;
they call intent-revealing functions that return the platform's existing domain types.

Module	Responsibility
mappers.ts	Row-to-domain mapping (Job, Freelancer, JobBid, DraftJob, TrackedProposal), category /
jobs.ts	experience / status coercion, relative-time and platform-fee helpers.
freelancers.ts	fetchOpenJobs, createJob, fetchClientProjects - the marketplace feed and job posting.
proposals.ts	fetchFreelancers (available talent), fetchSavedTalent, saveTalent, unsaveTalent.
profile.ts	fetchProposalsForJob, fetchMyJobBids, fetchMyProposals, fetchClientProposals, submitProposal (with milestone rows), updateProposalStatus, withdrawProposal.
	fetchProfileIds - resolves the signed-in user's client and freelancer row ids.

The session hook useCurrentUser was extended to expose userId, clientId and freelancerId alongside the
existing name, email and role, so any component can read live data scoped to the signed-in identity without
extra plumbing.

5. Live Wiring by Surface

Each surface was connected behind its existing props - the child components and their JSX were not modified,
eliminating visual regressions.

· Home marketplace (page.tsx). Open projects load live; the bid room fetches proposals per job; 'My Bids'
loads the freelancer's own proposals; posting a job inserts a real project; submitting, shortlisting, withdrawing
and accepting a bid now write to proposals with optimistic UI and failed-write fallback.

· Freelancer directory (FreelancerDirectory.tsx). Self-fetches available freelancers on mount while keeping
its original props.

· Client dashboard. Job posts, received proposals and saved talent now come from projects, proposals and
saved_talent for the signed-in client; KPI counts derive from the live rows.

· Freelancer dashboard. Recommended jobs load live; my proposals load from the database; the

quick-bid
modal submits a real proposal with the 7% fee and net-earnings calculation.

Contracts, the escrow wallet and the consultant listings remain on their existing mock data, as scoped for this sprint.

6. Security Posture

Every write continues to execute as the authenticated user under RLS; the application holds only the publishable anon key. New reads are additive and read-only:

Role	Access Introduced This Sprint
anon	SELECT on non-draft projects, clients and freelancers (public marketplace only). No writes.
authenticated - client	Own projects, proposals received on own projects, own saved-talent bookmarks.
authenticated - freelancer	Own proposals and their milestones, own bids in the drawer.
service role	Not used by the application; absent from the environment by design.

6.1 Admin User Addition (Implementation Note)

A dedicated administrator role was introduced to support platform operations without conflating staff with clients or freelancers. The schema adds a public.admins table keyed to public.users (uuid PK with cascade delete) and a SECURITY DEFINER function is_admin(uid uuid) used in RLS policies. Admins are granted read access to platform tables (users, clients, freelancers, projects, proposals, contracts, reviews, proposal_milestones) for moderation and overview purposes; no write policies were added and no service role is used in the application.

Role resolution checks the admins table in parallel with profile tables, so resolveUserRole returns "admin" w

The admin dashboard provides a platform overview (users, clients, freelancers, admins, profiles missing, projects/proposals/contracts counts), a searchable managed-users table showing type (client/freelancer/none) and admin markers, and recent activity (jobs/proposals). Navigation, notifications, and shell behavior were extended to accommodate the admin role while preserving existing client/freelancer flows. Administrators are provisioned out-of-band (SQL insert by email) and signup remains unchanged.

7. Quality Assurance & Verification

· Type safety: tsc --noEmit is clean across the new data layer and every rewired page.
· Lint: zero new findings from this sprint's files; the residual findings are the same pre-existing legacy

modal/footer warnings carried from earlier sprints.
· Production build: next build compiles and prerenders all routes successfully.
· No regressions: the public page, both dashboards and the post-job flow render and behave as before - now

backed by real rows.

Note: runtime verification against the hosted database is bounded by the environment, which holds only the anonymous
key. The follow-up migration must be applied in the Supabase SQL editor for names, proposal counts and anonymous
reads to resolve fully.

8. Work in Progress & Next Sprint Objectives

· Apply the follow-up and taxonomy-drop migrations on the hosted project and confirm the denormalized fields

populate for existing rows.
· Open the end-to-end hiring loop: award a proposal, create a real contract, and drive it through milestones.
· Move contracts and the escrow wallet/ledger off mock data onto persisted tables once the hiring loop exists.
· Add middleware for server-side session refresh and a sign-out control.
· Provision Storage buckets for KYC and portfolio uploads; enable Realtime for proposals and messaging.
· Integrate bKash and Nagad sandbox checkout with webhook-confirmed escrow locking.
· Re-introduce SMS-OTP and Google/Apple OAuth once external provider credentials are available.

9. Conclusion

MuktoKormo is no longer a mock interface. The marketplace feed, bidding flow and both role dashboards now
read and write live PostgreSQL rows through a typed, RLS-respecting data layer, with the design language and
component contracts preserved exactly. Security was strengthened without loosening ownership: new public
access is read-only, and denormalized fields were introduced precisely so the marketplace could be public
without exposing private tables. The next sprint turns this persistence into a complete hiring loop and begins
moving escrow onto real, persisted contracts.

Role model: the platform previously inferred identity purely from row presence - a client row made you a client, a freelancer row made you a freelancer. This sprint added a first-class administrator identity. An admin is deliberately not stored in either profile table, so an admin account has user type 'none': it is neither a client nor a freelancer.

Schema: migration 20261013000000_admin_role.sql adds a public.admins table (user_id primary key referencing public.users, with an optional note and created_at) backed by an RLS-enabled is_admin(uid) security-definer helper. Admins receive read-only policies across users, clients, freelancers, projects, proposals, contracts, reviews and proposal_milestones; no service-role key is used and no write surface was opened.

Access resolution: resolveUserRole now checks the admins table in parallel with the profile tables. On sign-in, useCurrentUser reports role 'admin' and dashboardPathForRole routes the session to /dashboard/admin. The client and freelancer dashboards redirect admins away, so role routing is unambiguous.

Admin dashboard (src/app/dashboard/admin): a tabbed administration console shows a platform overview (total users, open projects, total proposals, contracts and profile coverage), a searchable user-management table with type badges (client / freelancer / none) and an Admin marker, recent job posts and proposals for moderation, and contracts/escrow and dispute-queue panels - all read from live rows through the typed data layer.

Provisioning: administrators are created out-of-band. An account is promoted by inserting its public.users id into admins; the migration ships a commented snippet that promotes a user by email. Signup is unchanged - no admin option was added to the registration flow.

Verification: tsc --noEmit and ESLint pass with zero new findings; the migration is additive and idempotent against the existing seven-table schema, tables and policies.